Knowledge and Information Systems (2025) 67:1681–1711
https://doi.org/10.1007/s10115-024-02260-x

REGULAR PAPER

Improved affinity propagation clustering algorithms: a PSO-based approach

Ankita Sinha[1] · **Prasanta K. Jana**[2]

Received: 22 November 2023 / Revised: 29 June 2024 / Accepted: 7 October 2024 /
Published online: 9 November 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract
Traditional clustering algorithms such as K-means need to input the number of clusters before the start of the algorithm. Affinity propagation (AP) clustering algorithm solves this problem by considering each data point as a prospective cluster head (exemplar) and finding a set of appropriate exemplars by message passing. However, the AP clustering algorithm requires two parameters: preference and damping factor. Providing the parameters in advance poses the same issue faced in the traditional clustering algorithm. Moreover, all data points are not equally relevant for becoming cluster heads. To overcome these problems, we propose two parameter-free particle swarm optimization-based algorithms, PSO-AP$ver1$ and PSO-AP$ver2$. Furthermore, we introduce a novel version of mutant PSO where two cluster validity indices are used to judge the quality of the clustering solution. In PSO-AP$ver2$, we consider the internal data distribution using the square wave function to determine the initial preference value of data points. We conducted experiments on 8 real-world datasets to show the efficacy of our proposed algorithms over classic algorithms and two AP-based algorithms. We conducted the Friedman test followed by post hoc analysis to exhibit the significance of our work.

Keywords Data clustering · Affinity propagation · PSO · Influence function · Mutation

1 Introduction

Unsupervised categorization of unlabelled objects by identifying significant internal patterns in data is called clustering. Objects belonging to one cluster are identical to each other than objects belonging to other clusters. An abundance of algorithms have been proposed over the years to solve data clustering problem. Traditionally, clustering algorithms are mainly

✉ Ankita Sinha
ankita.2014dr0141@cse.ism.ac.in

Prasanta K. Jana
prasantajana@iitism.ac.in

[1] Department of Computer Science & Engineering, RRSDCE, Begusarai, Singhaul, Begusarai, Bihar 851134, India

[2] Department of Computer Science and Engineering, IIT(ISM) Dhanbad, Sardar Patel Nagar, Dhanbad, Jharkhand 826004, India

classified into two categories, hierarchical and partitional [1, 2]. These algorithms have been applied in a number of fields, such as image segmentation, web analysis, market subdivision, and many more [3]. The major drawback of these algorithms is that they require a prior understanding of the type of data and number of partitions, which is not available when dealing with real-world applications.

Thus, a significant paradigm change from conventional clustering to modern automatic clustering is needed, where the number of clusters is inherently adjusted and not subjected to prior knowledge. Automatic clustering algorithms have more chances of finding global optimal solutions, unlike traditional algorithms, which mostly search in local space and are governed by starting points [2]. However, the main focus of automatic clustering algorithms is to find a suitable number of clusters in a specified range [4]. This drawback is addressed by a novel automatic clustering approach called affinity propagation (AP) [5]. AP has gained immense popularity due to its following advantages. 1) It does not require the number of clusters as input, unlike K-means and K medoids. 2) Since this method is exemplar-based, there is no need to select a cluster centre in advance. 3) The square error of AP is less than contemporary algorithms [6]. 4) AP abstains from developing complex heuristics. Because of these advantages, AP has been successfully applied in various applications. A modified version of AP is used in ultrawideband channel [7]. Hu Huang-Shui et al. [6] used AP to avoid the need for a fixed number of clusters during the clustering process and showed increased energy efficiency in wireless sensor networks. Apart from this, AP has been applied for virtual machine placement in data centres [8], green cellular networks [9], indoor localization [10], intelligent fault detection [11], and many more.

The results of AP clustering depend on two critical parameters: preference and damping factor. Preference in affinity propagation (AP) sets initial exemplar preferences, while the damping factor controls message damping during iterations, influencing convergence speed. One of the main challenges in AP clustering is to determine the ideal preference value to produce an optimal number of exemplars depending on the dataset. To achieve an optimal number of clusters, it is recommended to run the algorithm multiple times with different preference values [5]. However, running the algorithm multiple times does not guarantee optimality. Apart from preference, choosing an appropriate damping factor is crucial to prevent numerical oscillations. Therefore, many improved algorithms [12–16] were proposed to address these challenges. For example, Li et al. [12] presented the algorithm adjustable preference affinity propagation clustering (APAP), which claims that every point does not have the same probability of becoming an exemplar. The algorithm suggests that each data point should be assigned a preference depending on the internal distribution of the data. It was experimentally shown that APAP provides better results than the original AP. However, it requires extra computational time and demands two additional parameters to be specified prior to algorithm execution. Many research works [17–20] used metaheuristic techniques to tune the parameters in AP. Jia et al. [17] proposed a novel technique called CAAP that uses the metaheuristic cuckoo search to find the optimal preference value. They countered the effect of preference initialization and eliminated numerical oscillation by increasing the damping factor gradually rather than setting it to a predefined value. However, the algorithm provides a common preference value for each data point. Zhou et al. [18] proposed a fuzzy clustering based on AP using density fruit fly optimization to optimize the parameters of AP. However, all such metaheuristic-based AP clustering approaches use a single cluster validity index (CVI) as a fitness function, which may not necessarily produce optimal results.

In this paper, we explore the use of multiple validity indices and propose two PSO-based AP clustering algorithms named PSO-AP$ver1$ and PSO-AP$ver2$. We introduce a novel mutant version of PSO, which uses two fitness functions and updates the best result in

each round by combining and comparing the results of both functions. A key feature of our algorithms is that they do not require the predefined value of preference and damping factors before the execution. The proposed PSO uses a combination of two CVIs, the silhouette score and Davies–Bouldin index (DBI), which are referred to as $primary$ function and $mutant$ function, respectively. PSO-AP$ver1$ considers the same preference for all data points. In the second approach, we consider internal data distribution using the square wave influence function [21, 22] to calculate the impact of a data point in its vicinity. The square wave influence function uses a parameter σ to decide the neighbourhood of a data point. The lower value of σ will accumulate all the points in one cluster, and the higher value will disperse all the points. Hence, it is very crucial to choose the correct value of σ. In PSO-AP$ver2$, we use our proposed mutant version PSO to decide the appropriate value of σ. The significant contributions of our proposed works are summarized as follows.

- We propose two PSO-based AP clustering algorithms, PSO-AP$ver1$ and PSO-AP$ver2$. Unlike most traditional and modern clustering algorithms, the proposed algorithms are parameter-free and do not conceive any predecided notion about data distribution or number of clusters. The algorithms automatically decide the number of partitions.
- A novel mutant version of PSO is proposed, which employs two fitness functions: $primary$ and $mutant$. The best solution in each iteration is updated by merging the results of both fitness functions.
- PSO-AP$ver2$ uses the internal distribution of data to assign different preference values. We use the square wave influence function to quantify the unique priority of each data point to become the cluster centre. The impact of a data point in its vicinity is calculated using this function, and accordingly, a unique preference is allocated to each data point.
- We performed extensive experiments on PSO-AP$ver1$ and PSO-AP$ver2$ for 8 validity indices and 8 datasets. The results of the proposed algorithms were compared with three standard algorithms: AP, K-means, and spectral clustering, as well as two AP-based algorithms, APAP and CAAP. The results demonstrate that the proposed algorithms work efficiently and outperform existing methods.

The remaining sections of this paper are as follows. Section 2 discusses some relevant related work. Background important to our proposed works is discussed in Sect. 3. We present our proposed works PSO-AP$ver1$ and PSO-AP$ver2$ in Sect. 4, followed by experimental results and analysis in Sect. 5. We conclude the paper in Sect. 6.

2 Related work

In this section, we discuss the most pertinent research studies related to the proposed algorithms. One of the first attempts to overcome the limitations of AP was suggested by Wang et al. [13], where they proposed adaptive affinity propagation (AAP), a technique to adjust the preference and damping factor. AAP begins with a high preference value (p), resulting in the formation of K clusters/exemplars, which are then checked for convergence. If the algorithm fails to converge, the preference value (p) is incrementally decreased by a small factor (ps), and the process is repeated. At the end of the execution, a set of clustering solutions is generated and evaluated using the silhouette score to determine the final solution. The major problem with this approach is that AAP starts with a very high p, which will generate a large number of clusters, whereas Frey et al. [5] states that p should be near to s_{median} to generate a moderate number of clusters. Moreover, AAP fails to achieve global

optima by scanning p with fixed step length. Jia et al. [17] employed the metaheuristic technique called cuckoo search to address the limitations of AAP. It was claimed that the cuckoo search also fails to calculate optimal results because of the fixed step length of levy flights. The authors proposed an improved cuckoo search (ICS) algorithm, where CAAP adapts the preference value (p) using the silhouette score as the fitness function. Zhou et al. introduced IFO-SAP [18], which utilizes a density fruit fly algorithm and pairwise constraints to construct a semi-supervised input matrix. An improved version of fruit fly optimization (IFO) was proposed to enhance the search capabilities of the fly. IFO was used to traverse through preference space and produced better results than AP and SAP [23]. AP based on optimization of preference (APBOP) uses an improved equilibrium optimizer and combines it with t-SNE [20]. In the first step, dimension reduction is applied, followed by a crisscross strategy to improve the hybrid equilibrium optimizer for an adaptive update of preference. APBOP uses the silhouette score as its objective function, similar to the other AP-based metaheuristic clustering techniques. However, the performance of APBOP was shown to be better than CAAP [17], IFO-SAP [18], and APAP [12]. Furthermore, all the above-discussed algorithms use metaheuristic techniques to tune the parameters of AP, which provides a probable list of clusters as output. A suitable CVI is selected as the fitness function to evaluate the quality of generated clusters. Silhouette score is used as the fitness function in all the above-mentioned algorithms. An abundance of CVIs [24] are available to check the quality of the solution, but none of them are entirely sufficient alone. Therefore, using just one validity index is neither justified nor adequate for validating the quality of clusters. Moreover, the data points are allocated the same preference irrespective of the internal distribution of data.

APAP [12] took into account internal data distribution to both initialize and repeatedly update the preference value. It initially sets preference value p_i to each data point $i \in D$, modified in each iteration until convergence. Two parameters, ϵ and τ, were introduced to assign data-specific preferences. It was experimentally proved that APAP provides better results than AP at the cost of increased time complexity. It should be noted that these two parameter values are predetermined and may not produce the optimal results for all types of data. Shang et al. [25] introduced a novel fast affinity propagation clustering approach (FAP), incorporating a new sampling algorithm to extract representative exemplars. The FAP further applies density-weighted spectral clustering, taking into account the internal data distribution to discover the final clustering solution. The authors experimentally showed superiority over basic AP and spectral clustering [26]. However, in the first stage of AP, the same preference value is assigned to each point, irrespective of the distribution of information contained in the data. Sun et al. [16] argues that the major drawback of AP clustering is its inefficiency in dealing with large-scale datasets. To overcome this, they proposed a computationally efficient two-stage algorithm. It applied AP to select only the potential exemplars by using a message-passing algorithm on an incomplete similarity matrix in the first stage and then applying the k nearest neighbour algorithm in the second stage to store only k nearest neighbours. Thus, it makes the algorithm computationally efficient, i.e. linear to the number of data points. Multi-exemplar affinity propagation (MEAP) [27] is a multi-exemplar model for AP, as a single exemplar model becomes inefficient for multiclass. They introduced the concept of super exemplar where each data point was assigned to it. This information was later used to identify a subclass of the category. The authors used max-sum belief propagation [28] to solve the NP-hard model and provide considerably decreased time complexity by using sparseness of the data. Gan & Ng [29] proposed a subspace clustering algorithm (SAP) which is an extension of AP [30]. The authors introduced an attribute weight in AP and iteratively updated it. The relative magnitude of attributes is used to identify subspaces for the clusters. However, changes in attribute weights affect the corresponding similarity matrix,

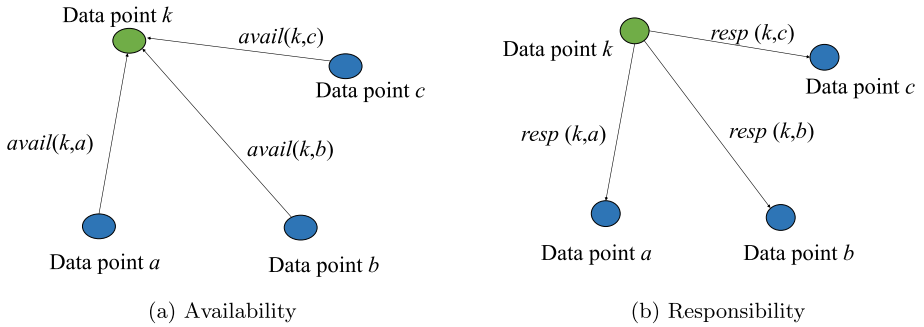


Fig. 1 Message passing in AP algorithm

necessitating its recalculation, unlike in the original AP, where the similarity matrix remains unchanged during execution. This constraint makes SAP inefficient in handling large datasets. Fujiwara et al. proposed two algorithms to address the problem of low clustering speed of AP [15, 31]. The authors proposed a technique in [15] to calculate each message's lower and upper bound to remove nonessential transfer of messages and obtain the same performance as the original AP. In K-AP, [32] authors proposed a way to generate a given number of optimal exemplars. However, this approach contradicts the basic advantage of AP, which helps to identify the internal data patterns without knowing the exact number of clusters. We present a comparative chart of significant AP-based clustering algorithms, including the proposed algorithms, in Table 1.

3 Background

Here, we briefly describe the working of the AP and PSO algorithms used to develop the proposed algorithms. The notations used in this manuscript are given in Table 2.

3.1 Affinity propagation

Input to AP is a pair of similarities $s(i,k)$ between a set of data points i and k, representing how nicely k can serve as an exemplar for i. The input similarity $s(i,j)$=$-||i-j||^2$ is set to negative squared Euclidean between i and k. Two types of messages, responsibility $resp(i,k)$ and availability $avail(i,k)$, are exchanged between data points as shown in Fig. 1. The data point k is a candidate exemplar. It sends availability message to the data points a, b and c, indicating how appropriately it can serve as a cluster head for these points (refer to Fig. 1a). In Fig. 1b, data points a, b, and c send responsibility messages to the exemplar k, representing how nicely they can serve as members of the cluster with k as cluster head.

Initially, $avail(i,k)$ is set to 0; $resp(i,k)$ and $avail(i,k)$ are calculated using Eqs. (1) and (2), respectively.

$$resp(i,k) = s(i,k) - max_{k' \text{ where } k' \neq k} avail(i,k') + s(i,k') \tag{1}$$

$$avail(i,k) = min\left\{0, resp(k,k) + \sum_{i' \notin \{i,k\}} max\left\{0, r(i',k)\right\}\right\} \tag{2}$$

Table 1 Comparative analysis of different AP-based clustering algorithms

Algorithm	Initialization Method	Fitness function	Convergence Criteria	Number of input Parameters	Key features	Disadvantages
CAAP	Random initialization of nests	Silhouette score	Minimum change in objective function or max iterations	High (parameters for CS, AP, and optimization process)	Combines cuckoo search (CS) with AP to optimize parameters	High computational cost due to the combination of CS and AP process
APBOP	Improved equilibrium optimizer initialization	Silhouette score	Minimum change in fitness value or max iterations	High (parameters for equilibrium optimizer, AP, and t-SNE)	Uses an improved equilibrium optimizer for optimizing preferences in AP, combined with t-SNE for visualization	Computationally expensive; t-SNE adds an additional layer of complexity and time
SAP	Semi-Supervised AP	NA	Initialization with labelled data	Medium (standard AP parameters plus semi-supervised learning inputs)	Integrates semi-supervised learning techniques with AP to incorporate prior knowledge in the clustering process	Requires labelled data, which may not always be available

Table 1 continued

Algorithm	Initialization Method	Fitness function	Convergence Criteria	Number of input Parameters	Key features	Disadvantages
PSO-APver1	Random initialization	Silhouette score and DBI	Minimum change in objective function or max iterations	Low (only parameters of PSO)	Proposed a mutant version of PSO with AP to optimize parameters. Uses a dual fitness function for better clustering result	High computational cost due to the combination of PSO and AP
PSO-APver2	Random initialization using square wave influence function	Silhouette score and DBI	Minimum change in objective function or max iterations	Low (only parameters of PSO)	Uses square wave influence function to assign data-specific preference value and combines proposed mutant PSO with AP to optimize parameters. Uses a dual fitness function for better clustering result	High computational cost due to the combination of PSO and AP.

Table 2 Notations

Notations	Description
D	Dataset
S	Similarity matrix
n	Number of data points in D
$resp(i,k)$	Responsibility(i,k)
$avail(i,k)$	Availability(i,k)
$s(i,j)$	Similarity between points i and j here $s(i,j)=-\|\|i-j\|\|^2$
s_{min}	Minimum of input similarity
s_{max}	Maximum of input similarity
s_{median}	Median of input similarity
p	Preference
σ	Neighbourhood radius
λ	Damping factor
$nPop$	Population size
nd	Number of variables optimized by PSO
P_i	Particle i
$P_i[pos]^t$	Position of particle i at iteration $t, \forall\, i \in nPop$
$P_i[vel]^t$	Velocity of particle i at iteration $t, \forall\, i \in nPop$
${pBest_i}^t$	Personal best of particle i till iteration $t, \forall\, i \in nPop$
$gBest$	Global fitness

These messages can be combined at any instant to get the set of exemplars and the points associated with them. The set of exemplars for a point i is identified by finding the value of k such that $avail(i,k) + resp(i,k)$ is maximum. If $i == k$, then i itself becomes an exemplar. The passing of messages $resp(i,k)$ and $avail(i,k)$ continues for a fixed number of iterations. The pseudo-code for AP is given in Algorithm 1.

Algorithm 1 :Affinity propagation algorithm

```
for all i, k in D do
    avail[i, k]=0
end for
for i =1 to itr do
    for all i, k in D do
        update the responsibility matrix using Eq. (1)
        update the availability matrix using Eq. (2)
    end for
end for
for all i, k in D do
        Find max{avail(i, k) + resp(i, k)}
    if i=k then
        exemplar.add(i)
    else
        i is exemplar for k
    end if
end for
```

3.2 Particle swarm optimization

PSO [33] provides a solution to the problem by taking a set of particles called population. Each particle is a candidate solution whose effectiveness is judged by a fitness function. The vectors used to define a particle P_i are given in Table 2. PSO also keeps track of the best fitness, $gBest$, in the entire population, which summarizes evolution. The particles move towards the global best in each iteration using past values. The position and velocity of a particle are updated using Eqs. (3) and (4).

$$P_i[vel]^t = w * P_i[vel]^{(t-1)} + C1 * r1()\left(pBest_i^t - P_i[Pos]^{t-1}\right) + C2 * r2\left(gBest - P_i[Pos]^{t-1}\right) \tag{3}$$

$$P_i[Pos]^t = P_i[Pos]^{t-1} + P_i[Vel] \tag{4}$$

Here, w is a coefficient which is incorporated in Standard PSO [34] and balances the exploration (global search) and exploitation (local search). C_1 and C_2 are constants representing cognitive and social factors, respectively. $r1$ and $r2$ are random numbers in the range [0, 1] that bring randomness to the calculation. The process of updating position and velocity is repeated for itr iterations. Algorithm 1 shows the steps of PSO.

Algorithm 2 Basic PSO

```
1: Initialize particles
2: for t = 1 to itr do
3:    for each particle P_i do
4:       fp = fitness(P_i[pos])
5:       if fp > fitness(pBest) then
6:          pBest = P_i[pos]
7:       end if
8:    end for
9:    gBest= Best P_i[pos] in all particles P
10:   for each particle P_i do
11:      Update velocity of P_i using equation 3
12:      Update position of P_i using equation 4
13:   end for
14: end for
```

4 Proposed work

This section presents the working of our proposed algorithms: PSO-AP$ver1$ and PSO-AP$ver2$.

4.1 PSO-AP*ver*1

We propose a mutant version of PSO and use it in our algorithms. This mutant PSO is modelled using the two essential steps, particle representation and fitness function, discussed in the following subsections.

4.1.1 Mutant version of PSO

PSO is used to optimize an objective function within a specified range to generate an optimal solution. The objective function is a mathematical representation of the problem to be optimized. It quantifies the quality or fitness of any given solution by using a fitness function, which directly evaluates the particle, or candidate solution, to guide the algorithm towards optimal output.

In the current approach, the particles themselves are not directly part of the objective function. Instead, the solutions generated by these particles are evaluated using cluster validity indices, which serve as the fitness function. Particles represent potential solutions in the clustering space, and each particle encodes a possible clustering solution. Rather than using the particles directly to determine their fitness, cluster validity indices are employed to assess the quality of clustering solutions. These indices measure the effectiveness of the clustering solution. Thus, the optimization process is guided by the fitness values derived from the cluster validity indices, which evaluate the performance of the solutions generated by the particles.

Using only one cluster validity index to evaluate the quality of a clustering solution is insufficient because different indices measure various aspects of clustering performance, leading to a potentially biased or incomplete assessment. Each index evaluates clustering quality based on distinct criteria, such as cohesion, separation, and inter-cluster similarity, and relying on a single index might favour certain cluster structures over others. Clustering performance can also be sensitive to data characteristics like the number of clusters, cluster shapes, densities, and sizes, which no single index can universally capture. Furthermore, each index has its strengths and weaknesses, making it necessary to combine indices to compensate for individual shortcomings. By incorporating multiple indices, the evaluation becomes more comprehensive, robust, and reliable, as the strengths of one index can balance the weaknesses of another, and a consensus-based approach can be adopted to provide a more balanced and accurate assessment of clustering quality. Therefore, we propose a mutant version of PSO that uses two fitness functions, $primary()$ and $mutant()$, to evaluate the clustering solution. This dual evaluation mechanism ensures a comprehensive optimization process by balancing different fitness criteria, ultimately aiming to find the most optimal solution.

The pseudo-code for mutant PSO is given in Algorithm 3. The algorithm takes the following inputs: C, Pop, $nPop$, $primary$, $mutant$, and t. Here, C is an array containing the clustering solution generated by running the AP algorithm on the initial population. Pop is the population of particles, and $nPop$ is the number of particles in the population. $primary$ and $mutant$ are the functions that calculate the primary and mutant fitness of a particle, respectively, and t is the number of iterations the algorithm will run. Initially, the $pBest$ of each particle is calculated using the $primary$ function, which is used to find the $gBest$. In the successive iterations, $gBest$ is updated using the following rule. The fitness of each particle is found using $primary$ and $mutant$ functions, and the best value is stored in $primary(pBest_i)$ and $mutant(pBest_i)$, respectively. Next, global best is updated using both $primary$ and $mutant$ functions and stored in $gBest_{primary}$ and $gBest_{mutant}$. The best particle's position is updated by comparing the positions of $gBest_{primary}$ and $gBest_{mutant}$. If the position of both are the same, then the algorithm continues. In contrast, if they are not equal, then a temporary position of $gBest_{temp}$ is calculated by taking the mean of positions of $gBest_{primary}$ and $gBest_{mutant}$. Now, $primary$ fitness and new fitness at the mean position are compared, and the best value fitness is stored in $gbest$ for the next iteration. The process

is continued for a fixed number of iterations or when there is a nominal change in the fitness function. Finally, we get an optimal value of p and λ.

Algorithm 3 : $mutantPSO(\mathrm{C}, Pop, nPop, primary, mutant, t)$

Require: $Pop, nPop, primary, mutant, t$
Ensure: optimal p and λ
1: **for** each particle P_i **do**
2: $fp =$ primary($C_[i]$)
3: **if** $fp >$ primary($pBest$) **then**
4: $pBest = P_i[pos]$
5: **end if**
6: **end for**
7: $gBest$= Best $P_i[pos]$ in all particles P
8: **for** itr in range(t) **do**
9: **for** i in range ($nPop$) **do**
10: $C[i]$ = AP($P_i[Pos]$)
11: $Pr[i]$ = primary($C[i]$)
12: $Mu[i]$ = mutant($C[i]$)
13: **if** $Pr[i] \geq pBest_{primary}[i]$ **then**
14: $pBest_{primary}[i] = Pr[i]$
15: **end if**
16: **if** $Mu[i] \leq pBest_{mutant}[i]$ **then**
17: $pBest_{mutant}[i] = Mu(i)$
18: **end if**
19: **if** Pos[$gBest_{primary}$] $\equiv$ Pos[$gBest_{mutant}$] **then**
20: $gBest = gBest_{primary}$
21: **else** mean=0.5*(Pos[$gBest_{primary}$] + Pos[$gBest_{mutant}$])
22: **if** $primary(mean) \geq gBest$ **then**
23: $gBest$=$primary(mean)$
24: **end if**
25: **end if**
26: **end for**
27: **for** i =1 range($nPop$) **do**
28: calculate $P_i.[vel]$ from Eq. (3)
29: update $P_i.[pos]$ using Eq. (4)
30: **end for**
31: **end for**

4.1.2 Particle representation

A vector of size $nPop$ * nd represents the particles; here, nd=2 (number of dimensions optimized) and $nPop$ is swarm size or the population size. The movement of all the particles is bound by an upper and lower value. A particle is allowed to choose p between s_{min} and s_{max} and λ in the range of 0.5 to 1 [5].

4.1.3 Fitness function

A plethora of validity indices are available to judge the quality of clustering solutions [24, 35]. The choice of validation methods should be independent of the algorithm. Cluster cohesion is an unsupervised cluster evaluation technique that measures the similarity of objects in the same cluster. On the contrary, cluster separation measures dissimilarity between the clusters.

In addition to cohesion and separation, CVIs are categorized as internal and external indexes. The internal indices assess cluster quality without using class labels, whereas external indices rely on true labels to evaluate clustering quality. In the real scenario, true labels are not available; therefore, we use internal validity measures to evaluate the clustering results.

Arbelaitz et al. [24] claimed that one internal evaluation criterion is not sufficient to establish the quality of generated clusters. They suggested combining internal and one or more external validity indices to judge the clustering results. We employ a combination of two internal validity measures: silhouette score and Davies–Bouldin index (DBI). It was experimentally exhibited in [24] that silhouette score generates better results than 30 other internal indices. Moreover, it was discovered that DBI also outperforms other indices and has a success score of more than 45%. Therefore, in our proposed works, we use silhouette score as the *primary* fitness function and DBI as a *mutant* function to maintain the diversity of the population and avoid being stuck in local optima.

- **Silhouette score**
  The silhouette coefficient combines both cohesion and separation. Silhouette coefficient for each point i is calculated as the ratio of difference of a_i and b_i by $max(b_i, a_i)$. Here, a_i is the average distance of I to all the data points in the same cluster, and b_i is the average dissimilarity of the point I with all data points in the next nearest cluster. The silhouette coefficient for a point i is given in Eq. (5).

$$\text{silhouette}_i = \frac{b_i - a_i}{max(b_i, a_i)} \tag{5}$$

  The value of the silhouette coefficient lies between [-1, 1]. A value close to 1 denotes a correct clustering solution. The silhouette coefficient of a cluster of n samples is calculated by taking the mean of all data points in dataset D as given in equation 6.

$$\text{silhouette} = \sum_{i=1}^{n} \frac{\text{silhouette}_i}{n} \tag{6}$$

- **Davies–Bouldin index (DBI)**
  DBI calculates the mean of similarity of the two most similar clusters. Here, similarity is measured as the fraction of distance within one cluster to the distance between different clusters. A lower DBI value indicates better-quality clusters. DBI is defined as follows.

$$DBI = \frac{1}{c} \sum_{x=1}^{c} max_{x \neq y} \left(\frac{\sigma_x + \sigma_y}{d_{xy}} \right) \tag{7}$$

  Here, c is the number of clusters, σ_x is the average distance of all points in the cluster from point x to its centroid, σ_y is the average distance of all points in the cluster with point y to its centroid, and d_{xy} is the distance between centroids of the cluster with point x and y.

4.1.4 Working of PSO-AP*ver*1

The PSO-AP*ver*1 algorithm is given in Algorithm 4. The algorithm finds optimal parameters p and λ for a given similarity matrix S. Initially, the algorithm generates a population of potential solutions, where each individual solution is characterized by values for p and λ. These values are randomly initialized within predefined bounds. The algorithm then uses these initial parameters to compute clustering results for each individual in the population

Algorithm 4 : PSO-AP$ver1$ algorithm

Require: Similarity matrix S
Ensure: Optimal p and λ
1: **for** i in range $(nPop)$ **do**
2: $\lambda_i = LB_\lambda + (UB_\lambda - LB_\lambda) \times random()$
3: $p_i = s_{min} + (s_{max} - s_{min}) \times random()$
4: **end for**
5: **for** i in range $(nPop)$ **do**
6: C[i]= AP(S, p_i, λ_i)
7: **end for**
8: (p_{opt},λ_{opt})= mutantPSO(C, Pop, $nPop$, $primary$, $mutant$, t)
9: C^*= AP(S, p_{opt}, λ_{opt})

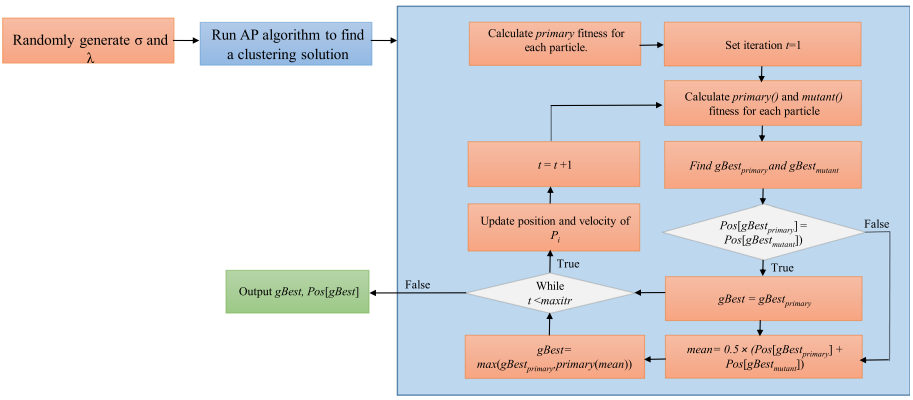


Fig. 2 Working of PSO-AP$ver1$ algorithm

using the AP clustering algorithm. Following this, the $mutantPSO$ function is called to improve the population of solutions. The $mutantPSO$ refines values of p and λ to optimize clustering performance. Finally, the optimal parameters (p_{opt} and λ_{opt}) are used to compute the final clustering solution C^* by invoking AP() method. The working of PSO-AP$ver1$ is illustrated in Fig. 2.

4.2 PSO-AP*ver*2

PSO-AP$ver2$ considers internal data distribution by using the square wave function, which computes a data point's effect in its neighbourhood. The influence function and the working of PSO-AP$ver2$ are as follows.

4.2.1 Influence function

In order to incorporate data-specific preference to each data point by considering internal data distribution, we employ the influence function [21, 22]. The impact of a data point x in its vicinity can be mathematically calculated using the influence function given in Eq. (8).

$$f_B^x(y) = f_B(y, x) \tag{8}$$

The notation $f_B^x(y) = f_B(y, x)$ represents a function transformation where the original function $f_B(y, x)$ is partially applied or fixed with respect to one of its variables. The function $f_B^x(y)$ takes two variables y and x, with some additional parameter or context denoted by B. $f_B(y, x)$ indicates that we are defining a new function which is derived from $f_B^x(y)$ by fixing the second argument x. It is a function of y only where x is treated as a constant parameter.

F^d is a d-dimensional feature space, and $x \in F^d$, n is the data points $\{x_1, x_2 ..., x_n\}$ in a feature vector D, where $D \subset F^d$. The influence of a data point $x \in F^d$ is a function $f_B^x : F^d \rightarrow R_+^0$ that is defined in terms of basic influence function f_B. The density function is the summation of the influence function of all the data points. Let $D = \{x_1, x_2 ..., x_N\} \subset F^D$, the density function is defined by Eq. (9).

$$f_b^D(x) = \sum_{i=1}^{N} f_B^{x_i}(x) \tag{9}$$

The above definitions of influence and density functions are arbitrary and depend on the distance function. A distance function $d : F^d \times F^d \rightarrow R_0^+$ defines the distance between two d-dimensional feature vectors. It finds the distance between two d-dimensional feature vectors. Many distance functions are available in the literature, such as Euclidean, Manhattan, cosine similarity, and Pearson correlation coefficient. Two basic influence functions are defined in the literature [21]: square wave function and Gaussian function. The square wave influence function is computationally efficient, provides clear boundaries, and simplifies implementation compared to the Gaussian function. Therefore, we use the square wave function in the proposed approach.

For a dataset D, having points x and y, the influence of x on y is estimated using the square wave influence function as shown in Eq. (10).

$$f_{square}(x, y) = \left\{ \begin{array}{cc} 0 & \text{if } d(x, y) > \sigma \\ 1 & \text{otherwise} \end{array} \right\} \tag{10}$$

Here, σ is a parameter that determines the neighbourhood of a point, and $d(x, y)$ represents the Euclidean distance between points x and y.

4.2.2 Particle representation

PSO-AP$ver2$ creates a vector of size $nPop*nd$ to initialize particles. The density of a point x can be calculated by following Eq. (11).

$$f_{square}^D(x) = \sum_{y=1}^{N} f_{square}(x, y) \tag{11}$$

By using the density function, preference of a point x is calculated as shown in Eq. (12).

$$p_x = s_{median} * f_{square}^D(x) \tag{12}$$

But, the value of p_x will go beyond the specified range of (s_{min}, s_{max}). Hence, it is normalized to lie in the required range. Our proposed mutant version of PSO is used to calculate the optimal value of σ in PSO-AP$ver2$.

4.2.3 Fitness function

Fitness function of PSO-AP$ver2$ is same as PSO-AP$ver1$ as discussed in Sect. 4.1.3.

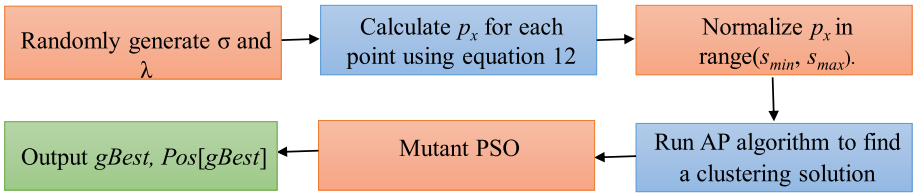


Fig. 3 Working of PSO-AP$ver2$ algorithm

4.2.4 Working of PSO-AP*ver*2

Algorithm 5 : PSO-AP$ver2$ algorithm

Require: Similarity matrix S
Ensure: optimal p and λ
1: **for** i in range ($nPop$) **do**
2: λ_i=LB_λ+($UB_\lambda - LB_\lambda$) × $random()$
3: σ_i=LB_σ+($UB_\sigma - LB_\sigma$) × $random()$
4: $p_i = s_{median} * f^D_{square}(x_i)$
5: $p_i = \frac{p_i - s_{min}}{(s_{max} - s_{min})}$
6: **end for**
7: **for** i in range ($nPop$) **do**
8: C[i]=AP(S, p_i,λ_i)
9: **end for**
10: (p_{opt},λ_{opt})=mutantPSO(C,Pop,$nPop$,$primary$,$mutant$,t)
11: C^*=AP(S,p_{opt},λ_{opt})

The PSO-AP$ver2$ algorithm given in Algorithm 5 enhances the original PSO-AP$ver1$ by introducing a new parameter σ and modifying the initialization process for the parameter p. The algorithm begins with the input similarity matrix S and aims to find the optimal values for parameters p and λ. Initially, for each individual in the population (size $nPop$), the algorithm randomly initializes λ within its lower and upper bounds (LB_λ and UB_λ) and σ within its respective bounds (LB_σ and UB_σ). The parameter p is then calculated using the median similarity value s_{median} and a domain-specific function $f^D_{square}(x_i)$, influencing the calculation of p. This calculated p is then normalized between s_{min} and s_{max}. After initializing these parameters, the algorithm computes the initial clustering results for each individual using the AP() method with the respective p and λ values. Subsequently, the $mutantPSO()$ process is employed to optimize the population iteratively, refining the values of p and λ by taking into account the clustering results. This process aims to identify the optimal parameters (p_{opt} and λ_{opt}) that provide the best clustering performance. Finally, the algorithm uses these optimal parameters to compute the final clustering solution C^* by invoking the AP() method. The working of PSO-AP$ver2$ is illustrated in Fig. 3.

4.3 Illustrative example

We apply PSO-AP$ver1$ to a toy dataset of 100 two-dimensional data points, as depicted in Fig. 4a. The algorithm is run for 30 iterations of PSO with a population size of 10 particles, which were allowed to roam in the range of (s_{min}, s_{max}) of the input dataset. Table 3 depicts the

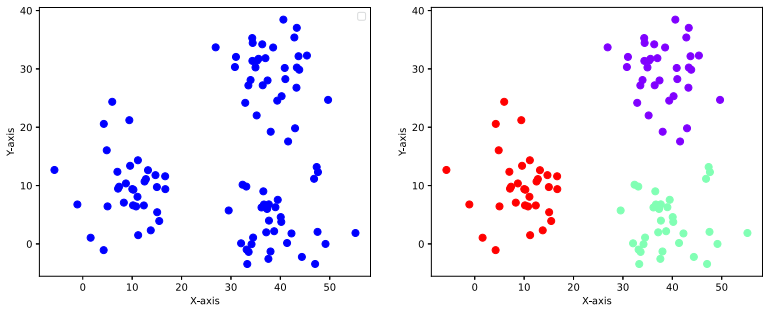


Fig. 4 Clustering on toy dataset

output of PSO-AP$ver1$ on the given dataset, preference generated, and corresponding fitness attained in every 10th iteration. The table shows that the first iteration's global best value of 0.819619506 is attained at a $preference$ value of − 330.3692371. The fitness value is in the range (− 0.010102567, 0.82), which further establishes the claim that incorrect selection of preference value can lead to meaningless results. Moreover, we observed that variations in $preference$ lie in the range (− 2616.06, − 330.369) and are refined in further iterations. The particles get the preference values in the range (− 1057.861842, − 265.506128) in the 10th iteration. PSO-AP$ver1$ keeps improving the preference to the optimal range. In 30th iteration, the range of $preference$ is (− 534.514,− 320.299), and the individual fitness value attained by most particles also comes very near to $gBest$ of 0.8417. The clustering result for the toy dataset is depicted in Fig. 4b.

4.3.1 Impact of PSO parameters on clustering

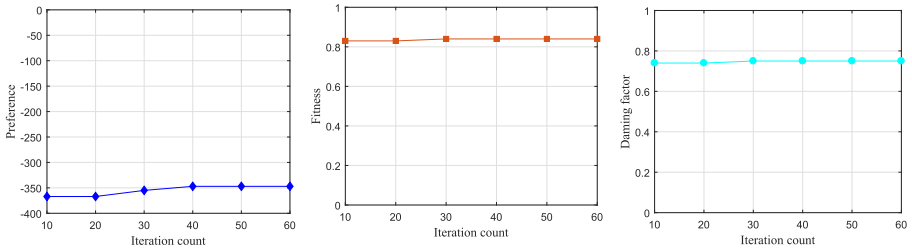
Figure 5 illustrates the impact of PSO parameters on clustering through six subfigures. Subfigures (a), (b), and (c) analyse the relationship between iteration count and preference, fitness, and damping factor, respectively. It is observed that the preference value in (a) remains relatively stable as iterations increase, fitness in (b) quickly converges to a high value and remains steady, and the damping factor in (c) remains constant throughout. Subfigures (d), (e), and (f) examine the influence of population size on preference, fitness, and damping factor. The preference value in (d) shows a slight decrease with increasing population size, while fitness in (e) remains high and stable across different population sizes, and the damping factor in (f) is unaffected by changes in population size. These findings suggest that the PSO algorithm is robust, with fitness and damping factor parameters showing stability across different iteration counts and population sizes, while the preference parameter exhibits minor sensitivity to population size.

4.4 Complexity analysis

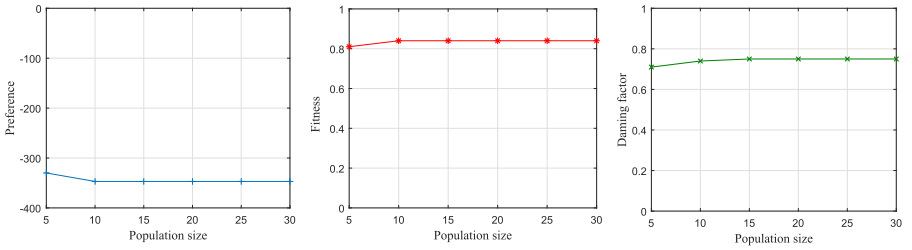
The time complexity of the *PSO-APver1* and *PSO-APver2* algorithms can be determined by analysing each step of the process. Initially, parameter initialization for each particle has a complexity of $O(nPop)$. The affinity propagation (AP) clustering is then applied to all particles, with a complexity of $O(nPop \times T_{AP})$. The key part of the algorithm involves optimizing

Table 3 Preference and fitness on toy dataset

Particles	Iteration 1		Iteration 10		Iteration 20		Iteration 30	
	p	*pBest*	*p*	*pBest*	*p*	*pBest*	*p*	*pBest*
P_1	− 1970.56	− 0.01	− 579.031	0.6536	−327.864	0.8249	− 350.756	0.8373
P_2	− 330.369	0.82	− 330.369	0.8196	− 330.369	0.8196	− 330.369	0.8196
P_3	− 2218.92	− 0	− 361.792	0.8332	− 361.792	0.8332	− 361.792	0.8332
P_4	− 2435.65	− 0.01	− 534.514	0.15	− 534.514	0.15	− 534.514	0.15
P_5	− 423.75	0.123	− 423.75	0.1232	− 209.518	0.425	− 328.244	0.8249
P_6	− 1057.86	0.303	− 1057.86	0.3028	− 355.243	0.8416	− 347.147	0.8417
P_7	− 2616.06	− 0.01	− 553.984	0.15	− 332.584	0.8248	− 332.584	0.8248
P_8	− 2318.85	− 0.01	− 553.208	0.1416	− 360.269	0.8366	− 360.269	0.8366
P_9	− 1162.03	0.301	− 367.888	0.8372	− 367.888	0.8372	− 367.888	0.8372
P_{10}	− 1189.72	0.305	− 265.506	0.3866	− 322.009	0.8373	− 320.299	0.8389



(a) Iteration count vs Preference (b) Iteration count vs Fitness (c) Iteration count vs Damping factor



(d) Population size vs Preference (e) Population size vs Fitness (f) Population size vs Damping factor

Fig. 5 Impact of PSO parameters on clustering

with *mutantPSO*, which has a complexity of $O(t \times nPop \times (T_{AP} + T_{primary} + T_{mutant}))$, where t is the number of iterations, T_{AP} is the time complexity of the AP function, $T_{primary}$ is the time complexity of computing primary fitness, and T_{mutant} is the time complexity of computing mutant fitness. Finally, the AP function is applied once more to obtain the optimal clustering solution, with a complexity of $O(T_{AP})$. Combining these steps, the overall time complexity of the algorithms are $O(nPop) + O(nPop \times T_{AP}) + O(t \times nPop \times (T_{AP} + T_{primary} + T_{mutant})) + O(T_{AP})$. Given that the optimization step typically dominates, the dominant term is $O(t \times nPop \times (T_{AP} + T_{primary} + T_{mutant}))$ making this the primary time complexity of the algorithms using *mutantPSO*.

5 Experimental results and analysis

We conducted several tests to evaluate the performance of the proposed algorithms. We used eight real-world datasets as given in Table 4. These are popular benchmark datasets for the evaluation of clustering and are available publicly at the UCI machine learning repository [36]. The experiments were performed on a 2.60 GHz Intel Core i7-4510U processor, 8 GB RAM, and Ubuntu 20.04.1 LTS operating system. The code was written in Python 3.11.5. The results of the proposed methods, PSO-AP$ver1$ and PSO-AP$ver2$, were compared with three standard algorithms: AP [37], K-means [3], and spectral clustering (SC) [26]. For K-means and SC, the number of clusters is required as a parameter. To ensure a fair comparison, we provided the actual number of clusters as input for these algorithms. We also compared the proposed algorithms with two AP-based clustering methods. We used 8 cluster validity indices to test the efficacy of our proposed works, which are discussed as follows.

Table 4 Datasets description

Dataset	Size	Class	Dimensions
Seeds	210	3	7
Wine	178	3	13
Haberman	306	2	3
Ionosphere	351	2	34
User knowledge	403	4	5
Balance	625	3	4
Glass	214	7	9
E coli	336	8	7

V-measure is an external entropy-based clustering evaluation metric [38]. The metric is weighted to prioritize completeness and homogeneity. It is calculated using Eq. (13).

$$V_\beta = \frac{(1+\beta) * h * c}{(\beta * h) + c} \tag{13}$$

Here, β is a constant set to favour either homogeneity or completeness. In the computation, completeness is given more weight if β is greater than 1, and homogeneity is given more weight if β is less than 1. The principal benefit of this assessment metric is its independence from the number of class labels, clusters, data size, and clustering algorithm.

The Fowlkes–Mallows index (FMI) is a tool to assess the similarity of different clustering methods [39]. It evaluates a clustering algorithm's performance by comparing its results to the ground truth or ideal clusters as given in Eq. (14).

$$\text{FMI} = \sqrt{\left(\frac{\text{TP}}{\text{TP}+\text{FP}}\right)\left(\frac{\text{TP}}{\text{TP}+\text{FN}}\right)} \tag{14}$$

Here, TP (true positive) denotes the number of points that are part of the same clusters in both predicted and true labels. The number of points belonging to the same clusters in true labels but not in predicted labels is known as FP (false positive). FN (false negative) is the quantity of points that are part of the same clusters in the predicted labels but not in the true labels.

Rand index measures similarity in clustering results produced by two different clustering algorithms. Rand index is computed by Eq. (15).

$$\text{Rand Index(RI)} = \frac{\text{Number of pairwise correct predictions}}{\text{Total number of possible pairs}} \tag{15}$$

The adjusted Rand index (ARI) is a modified version of the rand index that measures the similarity between two clusters by using Eq. (16).

$$\text{ARI} = \frac{\text{RI} - \text{Expected(RI)}}{\max(\text{RI}) - \text{Expected(RI)}} \tag{16}$$

Calinski–Harabasz (CH) score measures the similarity of an object to its own cluster in comparison with its separation from other clusters. Equation (17) is used to measure the CH index for a dataset of k clusters.

$$\text{CH}(k) = \frac{B(k)}{W(k)} \times \frac{n-k}{k-1} \tag{17}$$

where n and k stand for data points and clusters, respectively, and $W(k)$ and $B(k)$ indicate within and between cluster variance. The quality of the clustering increases with the index level.

Adjusted mutual information evaluates the consensus between true values and the predicted values. For two clusters X and Y, it is defined in Eq. (18).

$$\mathrm{AMI}(X, Y) = \frac{\mathrm{MI}(X, Y) - E(\mathrm{MI}(X, Y))}{\mathrm{avg}(H(X), H(Y)) - E(\mathrm{MI}(X, Y))} \tag{18}$$

Here, $\mathrm{MI}(X, Y)$ is the mutual information (MI) between X and Y, whereas $E(\mathrm{MI}(X, Y))$ is the expected MI between them. $H(X)$ and $H(Y)$ are entropy of clusters X and Y, respectively. Apart from this, we also compared the results using the silhouette score and DBI, which are discussed in Sect. 4.1.3.

5.1 Comparison with baseline clustering algorithms

We compared the proposed algorithms with three standard clustering algorithms: AP, k-means, and spectral clustering. Figures 6 and 7 present DBI and silhouette score for the eight datasets of all the five clustering algorithms. It can be observed from the figures that both the values are comparable and in the same range. All algorithms provide better results for Seeds, Glass, E coli, and Wine datasets, whereas they are not as good for the remaining datasets. Spectral clustering generated a particularly poor solution for the wine dataset, as the graph created during the process was disconnected. It is evident from the results that the performance of PSO-AP$ver1$ and PSO-AP$ver2$ is better than all other standard algorithms. The V-measure results for all datasets are illustrated in Fig. 8. The value of β, which balances the homogeneity and completeness in the V-measure, was set to 0.6. The results show that PSO-AP$ver1$ and PSO-AP$ver2$ are at par with the standard algorithms. The Rand index and FMI for all the datasets are given in Figs. 9 and 10, respectively. CH score, ARI, and AMI are given in Tables 5, 6 and 7, respectively.

k-means was provided with the actual number of clusters as the parameter, resulting in better performance compared to the standard AP for most datasets and CVIs. SC's results are inconsistent with other algorithms, and its quality greatly depends on the dataset. For some datasets like Ionosphere, Glass, and Haberman, SC performs better, but its performance is poor for the remaining datasets. The analysis of results demonstrates that PSO-AP$ver1$ and PSO-AP$ver2$ perform much better than standard AP, k-means, and SC. The reason is that the proposed algorithms use PSO to optimize the AP parameters preference and damping factor. Moreover, PSO-AP$ver2$ considers the internal data distribution to assign the initial preferences to the data points. Therefore, it is evident from the results that PSO-AP$ver2$ is better in comparison with PSO-AP$ver1$.

5.2 Comparison with AP-based clustering algorithms

We compared the results of the proposed PSO-AP$ver1$ and PSO-AP$ver2$ with two standard AP-based clustering algorithms, APAP [12] and CAAP [17]. APAP independently determines initial preference values based on data distribution and automatic adjustments during subsequent iterations. CAAP introduces an improved cuckoo search algorithm to refine affinity propagation (AP) clustering by addressing issues such as oscillations and optimizing present preference values. We ran all four algorithms for the eight real-world datasets and measured the performance with respect to seven validity metrics. The test results with respect to CH

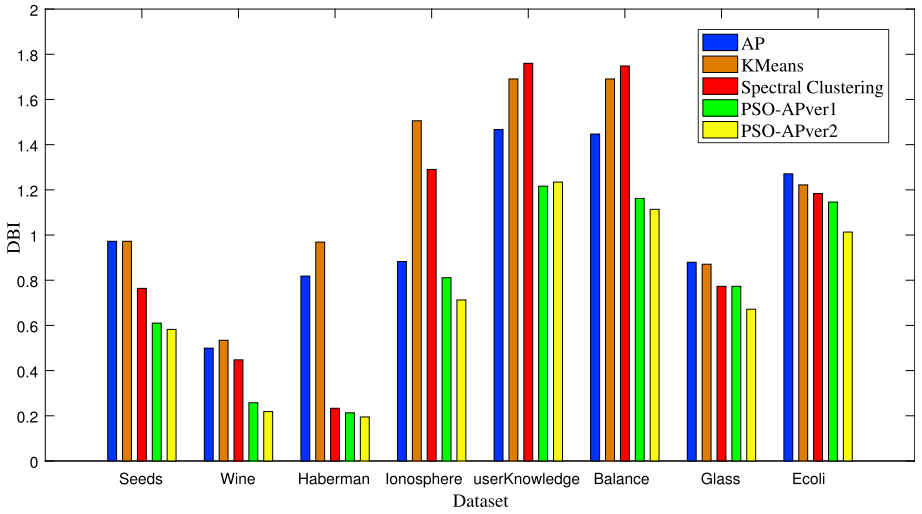


Fig. 6 DBI

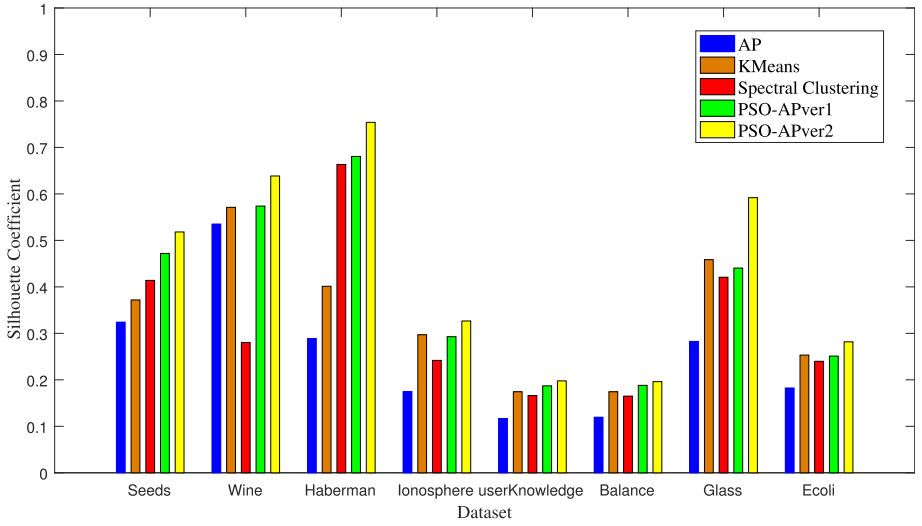


Fig. 7 Silhouette score

score, FMI, V-measure, silhouette score, rand score, ARI, and AMI are shown in Tables 8, 9, 10, 11, 12, 13, 14, respectively. It can be noticed from the results that the CH score is highest for the PSO-AP$ver2$, followed by PSO-AP$ver1$ for all the datasets, which shows superiority over the APAP and CAAP. Similar outcomes are observed across the other validity metrics, as evident from the results. APAP and CAAP also do not converge for some datasets such as Haberman, User Knowledge, Balance, and E coli; therefore, we inserted a dash (–) in the tables for such datasets. Thus, we conclude that PSO-AP$ver1$ and PSO-AP$ver2$ show significantly better performance than APAP and CAAP.

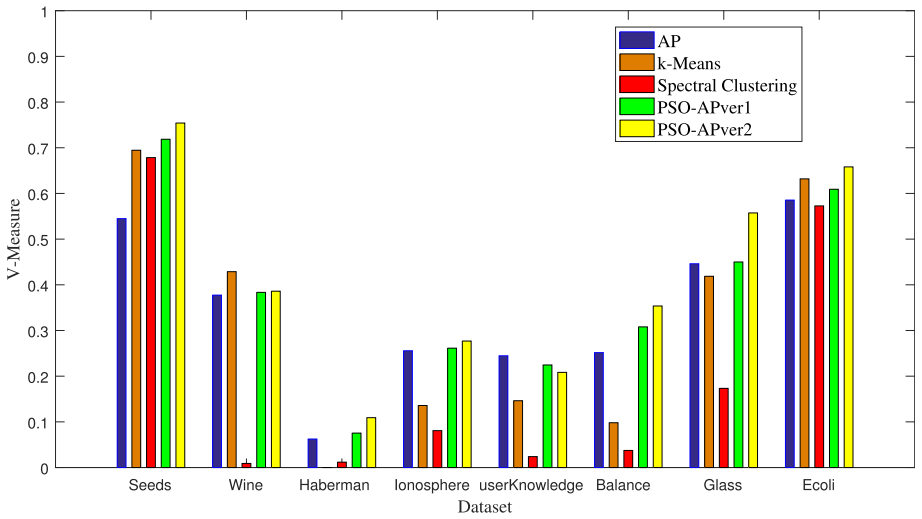


Fig. 8 V-measure

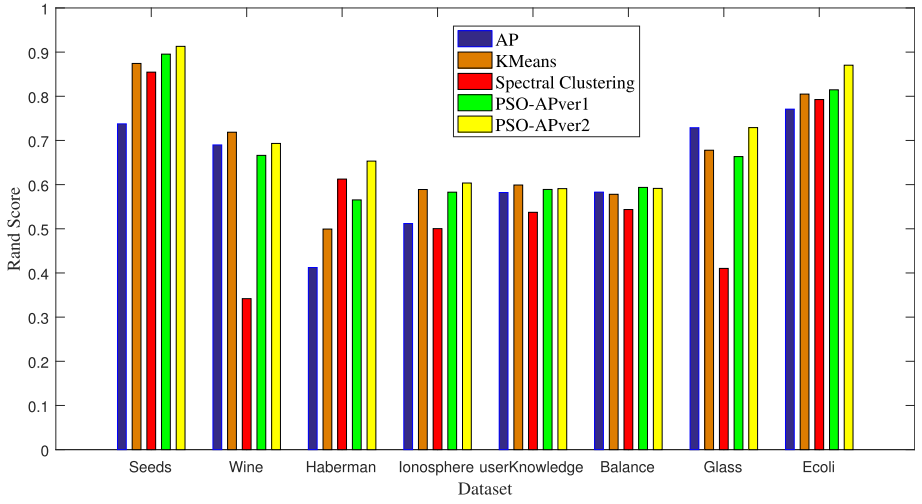


Fig. 9 Rand score

5.3 Statistical tests

We employ the nonparametric Friedman test [40] to demonstrate the significant difference between the performances of these five algorithms. We considered three clustering validity indices for this test: CH score, silhouette coefficient, and ARI. The null hypothesis states that the mean clustering performance of all five algorithms is the same, and the alternative hypothesis states that there is a significant difference in the clustering performance of the algorithms. The significance level for this test is set to 0.05. It generates a rank for each dataset instance with respect to each algorithm as depicted in Table 15. The test results are

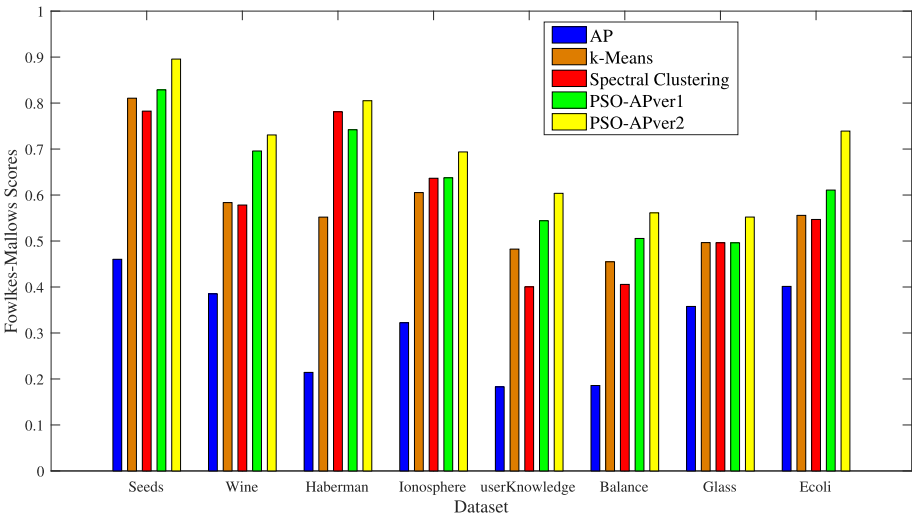


Fig. 10 Fowlkes–Mallows scores

Table 5 Calinski–Harabasz score

	AP	k-means	SC	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	265.916765	375.804961	361.325040	385.613679	398.613679
Wine	1282.275313	561.815658	3.369103	1285.379596	2063.082055
Haberman	176.468665	239.926360	14.117834	246.743410	265.830438
Ionosphere	20.578545	120.157883	53.122518	132.106740	149.681446
User knowledge	77.31683	136.8272	127.7598	145.8932	166.8532
Balance	75.96378	136.8272	127.7034	143.8164	158.4745
Glass	105.401237	114.004985	31.083903	119.276653	123.240236
E coli	96.718767	149.645153	118.266788	149.665265	159.497197

Table 6 Adjusted Rand index

	AP	k-means	SC	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.277259	0.716620	0.673380	0.879900	0.892676
Wine	0.190512	0.371114	0.000351	0.197614	0.257614
Haberman	0.014268	− 0.001813	0.011577	0.022607	0.018796
Ionosphere	0.080806	0.177607	− 0.046451	0.196237	0.229610
User knowledge	0.036925	0.160496	0.030619	0.182016	0.237311
Balance	0.038669	0.116294	0.042624	0.141563	0.197329
Glass	0.188669	0.264108	0.067310	0.250800	0.291306
E coli	0.238424	0.420597	0.408629	0.486229	0.650489

Table 7 Adjusted mutual information

	AP	k-means	SC	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.480561	0.692216	0.676522	0.777003	0.831578
Wine	0.331545	0.422687	0.000185	0.339560	0.395549
Haberman	0.031897	− 0.002522	0.008269	0.053959	0.088569
Ionosphere	0.180418	0.133008	0.084028	0.227705	0.248575
User Knowledge	0.186683	0.140347	0.02027	0.203986	0.25071
Balance	0.038669	0.116294	0.042624	0.141563	0.197329
Glass	0.342549	0.394030	0.140557	0.429175	0.459696
E coli	0.505690	0.594379	0.538901	0.588343	0.634762

Table 8 Calinski–Harabasz score

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	92.748851	268.281878	385.6137	398.613679
Wine	123.673219	153.671917	1285.38	2063.082055
Haberman	199.649839	–	246.7434	265.830438
Ionosphere	45.895023	20.373286	132.1067	149.681446
User Knowledge	–	36.108045	145.8932	166.8532
Balance	113.255605	–	143.8164	158.4745
Glass	8.858139	102.309494	119.2767	123.240236
E coli	–	99.987880	149.6653	159.497197

Table 9 Fowlkes–Mallows score

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	92.748851	268.281878	385.6137	398.613679
Wine	123.673219	153.671917	1285.38	2063.082055
Haberman	199.649839	–	246.7434	265.830438
Ionosphere	45.895023	20.373286	132.1067	149.681446
User Knowledge	–	36.108045	145.8932	166.8532
Balance	113.255605	–	143.8164	158.4745
Glass	8.858139	102.309494	119.2767	123.240236
E coli	–	99.987880	149.6653	159.497197

as follows: χ_r^2 statistic value 79.7667 and p value $<.00001$ for degree of freedom 4. The p value is less than the significant level of 0.05, which rejects the null hypothesis.

Wilcoxon signed-rank test is used to perform pairwise comparisons of the algorithms. The pairwise test results at a significance level of 0.05 are shown in Table 16. It is evident from the results that the p value for each test instance is lesser than the significance level (<0.05). Therefore, we can conclude that each pair of algorithms significantly differs in performance.

Table 10 V-measure

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.588456	0.543769	0.718622	0.754067
Wine	0.722115	0.516638	0.383443	0.386025
Haberman	0.104757	–	0.075393	0.109138
Ionosphere	0.270738	0.261964	0.261286	0.276879
User Knowledge	–	0.402345	0.224529	0.208345
Balance	0.231609	–	0.307918	0.353641
Glass	0.448777	0.421915	0.44996	0.557313
E coli	-	0.596597	0.609141	0.658098

Table 11 Silhouette score

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.370958	0.318170	0.471934	0.518139
Wine	0.236309	0.267458	0.573811	0.63855
Haberman	0.326200	–	0.680647	0.753946
Ionosphere	0.227122	0.196267	0.292842	0.326592
User Knowledge	–	0.187632	0.187099	0.197748
Balance	0.187211	–	0.188116	0.196081
Glass	0.472482	0.264315	0.440503	0.591913
E coli	–	0.190721	0.251066	0.281784

Table 12 Rand score

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.774846	0.732194	0.895443	0.913049
Wine	0.966101	0.902939	0.666222	0.693265
Haberman	0.396549	–	0.565402	0.653252
Ionosphere	0.580170	0.534505	0.582906	0.60363
User Knowledge	–	0.527686	0.589097	0.590913
Balance	0.591015	–	0.593703	0.591462
Glass	0.6778991	0.727962	0.66342	0.729084
E coli	–	0.776990	0.814677	0.870398

Table 13 Adjusted Rand score

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.405585	0.254598	0.8799	0.892676
Wine	0.009275	− 0.002548	0.197614	0.257614
Haberman	0.004374	–	0.022607	0.018796
Ionosphere	0.191653	0.118599	0.196237	0.22961
User Knowledge	–	0.106642	0.182016	0.237311
Balance	0.065662	–	0.141563	0.197329
Glass	0.277401	0.153278	0.2508	0.291306
E coli	–	0.265053	0.486229	0.650489

Table 14 Adjusted mutual index

	APAP	CAAP	PSO-AP$ver1$	PSO-AP$ver2$
Seeds	0.535652	0.475783	0.777003	0.831578
Wine	0.018579	− 0.008508	0.33956	0.395549
Haberman	0.022687	–	0.053959	0.088569
Ionosphere	0.234348	0.187681	0.227705	0.248575
User Knowledge	–	0.315977	0.203986	0.25071
Balance	0.189243	–	0.141563	0.197329
Glass	0.413178	0.311091	0.429175	0.459696
E coli	–	0.520351	0.588343	0.634762

Table 15 Rank matrix

AP	K-means	SC	PSO-AP$ver1$	PSO-AP$ver2$
1	3	2	4	5
3	2	1	4	5
2	3	1	4	5
1	3	2	4	5
1	3	2	4	5
1	3	2	4	5
2	3	1	4	5
1	3	2	4	5
1	2	3	4	5
2	3	1	4	5
1	2	3	4	5
1	4	2	3	5

Table 15 continued

AP	K-means	SC	PSO-AP$ver1$	PSO-AP$ver2$
1	3	2	4	5
1	3	2	4	5
1	4	2	3	5
1	4	2	3	5
1	3	2	4	5
2	5	1	3	4
3	1	2	5	4
2	3	1	4	5
2	3	1	4	5
1	3	2	4	5
2	4	1	3	5
1	3	2	4	5

6 Conclusion and future work

This paper presents two PSO-based AP clustering algorithms, PSO-AP$ver1$ and PSO-AP$ver2$, to handle the drawbacks of traditional data clustering algorithms. We have proposed a novel mutant version of PSO having two cluster validity indices as fitness functions to evaluate the clustering solutions and avoid being stuck in local optima. We also considered the internal distribution of data and assigned unique preferences to each data point using the square wave influence function in PSO-AP$ver2$. The experimental results showed that PSO-AP$ver1$ and PSO-AP$ver2$ produce better clustering solutions than existing clustering approaches. The results were also statistically validated using the Friedman and Wilcoxon signed-rank tests. The time complexity of the proposed approach increases because of the usage of two fitness functions. PSO-AP clustering can be applied to several recent applications, such as enhancing blockchain consensus mechanisms, analysing gene expression data, and segmenting grayscale images. In future extensions of this work, we aim to enhance clustering quality while reducing time complexity.

Table 16 Wilcoxon signed-rank test

	AP-PSO-AP$ver1$	AP-PSO-AP$ver2$	K-means-PSO-AP$ver1$	K-means-PSO-AP$ver2$	SC-PSO-AP$ver1$	SC-PSO-AP$ver2$	PSO-AP$ver1$- PSO-AP$ver2$
n	24	24	24	24	24	24	24
Ranks	(– 24, + 0, 0)	(– 24, + 0, 0)	(– 19, + 5, 0)	(– 23, + 1, 0)	(– 24, + 0, 0)	(– 24, + 0, 0)	– 23, + 1, 0
Mean rank	− 12.50 + NAN	− 12.50 + NAN	− 14.11 + 6.40	− 12.57 + 11.00	− 12.50 + NAN	− 12.50 + NAN	− 13.00 +1.00
Z value	− 4.29	− 4.29	− 3.37	− 3.97	− 4.29	− 4.29	− 4.26
p-value (2-tailed)	0.000	0.000	0.001	0.000	0.000	0.000	0.000

Acknowledgements This research work is supported by Rashtrakavi Ramdhari Singh Dinkar College of Engineering (RRSDCE), Begusarai, under the Department of Science, Technology and Technical Education, Government of Bihar, and Department of Computer Science and Engineering IIT(ISM), Dhanbad. The authors thank the Department of Science, Technology and Technical Education, Government of Bihar, and IIT(ISM) Dhanbad, for their research support.

Author Contributions Both authors performed conceptualization and problem formulation. Ankita Sinha designed the algorithm, performed experiments, and validated the results. Both authors edited vigorously to improve the final manuscript.

Data availability statement No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors affirm that they have no affiliations or conflict of interest that could have affected the research presented in this paper.

References

1. Han J, Pei J, Kamber M (2011) Data mining: concepts and techniques. Elsevier, Amsterdam
2. Ezugwu AE, Ikotun AM, Oyelade OO, Abualigah L, Agushaka JO, Eke CI, Akinyelu AA (2022) A comprehensive survey of clustering algorithms: state-of-the-art machine learning applications, taxonomy, challenges, and future research prospects. Eng Appl Artif Intell 110:104743
3. Jain AK (2010) Data clustering: 50 years beyond K-means. Pattern Recogn Lett 31(8):651–666
4. Jose-Garcia A, Gomez-Flores W (2016) Automatic clustering using nature-inspired metaheuristics: a survey. Appl Soft Comput 41:192–213
5. Frey BJ, Dueck D (2007) Clustering by passing messages between data points. Science 315(5814):972–976
6. Huang-Shui H, Yu-Xin G, Chu-Hang W, Dong G (2022) Affinity propagation and chaotic lion swarm optimization based clustering for wireless sensor networks. IEEE Access 10:71545–71556
7. Lee JY (2023) A clustering technique for ultra-wideband channel using affinity propagation. IEEE Antennas Wirel Propag Lett 22:1818–1822
8. Li Z, Guo S, Yu L, Chang V (2020) Evidence-efficient affinity propagation scheme for virtual machine placement in data center. IEEE Access 8:158356–158368
9. Lee SH, Sohn I (2015) Affinity propagation for energy-efficient BS operations in green cellular networks. IEEE Trans Wirel Commun 14(8):4534–4545
10. Yaro AS, Maly F, Maly K (2023) Improved indoor localization performance using a modified affinity propagation clustering algorithm with context similarity coefficient. IEEE Access 11:57341–57348
11. Hu Q, Zhang Q, Si XS, Sun GX, Qin AS (2020) Intelligent fault diagnosis approach based on composite multi-scale dimensionless indicators and affinity propagation clustering. IEEE Sens J 20(19):11439–11453
12. Li P, Ji H, Wang B, Huang Z, Li H (2017) Adjustable preference affinity propagation clustering. Pattern Recogn Lett 85:72–78
13. Wang K, Zhang J, Li D, Zhang X, Guo T (2008) Adaptive affinity propagation clustering. arXiv preprint arXiv:0805.1096
14. Lu Z, Carreira-Perpinan MA (2008) Constrained spectral clustering through affinity propagation. In: 2008 IEEE conference on computer vision and pattern recognition, pp 1–8. IEEE
15. Fujiwara Y, Irie G, Kitahara T (2011) Fast algorithm for affinity propagation. In: Twenty-second international joint conference on artificial intelligence
16. Sun L, Guo C, Liu C, Xiong H (2017) Fast affinity propagation clustering based on incomplete similarity matrix. Knowl Inf Syst 51(3):941–963
17. Jia B, Yu B, Wu Q, Wei C, Law R (2016) Adaptive affinity propagation method based on improved cuckoo search. Knowl-Based Syst 111:27–35
18. Zhou R, Liu Q, Wang J, Han X, Wang L (2021) Modified semi-supervised affinity propagation clustering with fuzzy density fruit fly optimization. Neural Comput Appl 33:4695–4712
19. Liu Y, Liu J, Jin Y, Li F, Zheng T (2020) An affinity propagation clustering based particle swarm optimizer for dynamic optimization. Knowl-Based Syst 195:105711

20. Duan Y, Liu C, Li S, Guo X, Yang C (2023) An automatic affinity propagation clustering based on improved equilibrium optimizer and t-SNE for high-dimensional data. Inf Sci 623:434–454
21. Hinneburg A, Keim DA (1998) An efficient approach to clustering in large multimedia databases with noise. In: KDD, vol 98, pp 58–65
22. Zhang A (2006) Advanced analysis of gene expression microarray data, vol 1. World Scientific, Singapore
23. Yu XIAO, Jian Yu (2008) Semi-supervised clustering based on affinity propagation algorithm. J Softw 19(11):2803–2813
24. Arbelaitz O, Gurrutxaga I, Muguerza J, Perez JM, Perona I (2013) An extensive comparative study of cluster validity indices. Pattern Recogn 46(1):243–256
25. Shang F, Jiao LC, Shi J, Wang F, Gong M (2012) Fast affinity propagation clustering: a multilevel approach. Pattern Recogn 45(1):474–486
26. Ng AY, Jordan MI, Weiss Y (2002) On spectral clustering: analysis and an algorithm. In: Advances in neural information processing systems, pp 849–856
27. Wang CD, Lai JH, Suen CY, Zhu JY (2013) Multi-exemplar affinity propagation. IEEE Trans Pattern Anal Mach Intell 35(9):2223–2237
28. Kschischang FR, Frey BJ, Loeliger HA (2001) Factor graphs and the sum-product algorithm. IEEE Trans Inf Theory 47(2):498–519
29. Gan G, Ng MKP (2015) Subspace clustering using affinity propagation. Pattern Recogn 48(4):1455–1464
30. Vidal R (2011) Subspace clustering. IEEE Signal Process Mag 28(2):52–68
31. Fujiwara Y, Nakatsuji M, Shiokawa H, Ida Y, Toyoda M (2015) Adaptive message update for fast affinity propagation. In: Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining, pp 309–318. ACM
32. Zhang X, Wang W, Norvag K, Sebag M (2010) K-AP: generating specified K clusters by efficient affinity propagation. In: 2010 IEEE international conference on data mining, pp 1187–1192. IEEE
33. Kuila P, Jana PK (2014) Energy efficient clustering and routing algorithms for wireless sensor networks: particle swarm optimization approach. Eng Appl Artif Intell 33:127–140
34. Shi Y, Eberhart R (1998) A modified particle swarm optimizer. In: 1998 IEEE international conference on evolutionary computation proceedings. In: IEEE world congress on computational intelligence (Cat. No. 98TH8360), pp 69–73. IEEE
35. Maulik U, Bandyopadhyay S (2002) Performance evaluation of some clustering algorithms and validity indices. IEEE Trans Pattern Anal Mach Intell 24(12):1650–1654
36. https://archive.ics.uci.edu/
37. Dueck D (2009) Affinity propagation: clustering data by passing messages. University of Toronto, Toronto
38. Rosenberg A, Hirschberg J (2007) V-measure: a conditional entropy-based external cluster evaluation measure. In: Proceedings of the 2007 joint conference on empirical methods in natural language processing and computational natural language learning (EMNLP-CoNLL), pp 410–420
39. Fowlkes EB, Mallows CL (1983) A method for comparing two hierarchical clusterings. J Am Stat Assoc 78(383):553–569
40. Scheff SW (2016) Fundamental statistical principles for the neurobiologist: a survival guide. Academic Press, Cambridge

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Ankita Sinha is currently working as an Assistant Professor in the Department of Computer Science & Engineering at Rashtrakavi Ramdhari Singh Dinkar College of Engineering, Begusarai, Bihar, India, under the Science, Technology and Technical Education Department Government of Bihar, India. She did her PhD and MTech from the Department of Computer Science & Engineering at Indian Institute of Technology (ISM) Dhanbad, India. Her research interests include data clustering, parallel programming, and big data.



Prof. Prasanta K. Jana received an MTech degree in Computer Science from the University of Calcutta in 1988 and PhD in 2000 under the joint supervision of the Indian Statistical Institute and Jadavpur University, Kolkata. Currently, he is a Professor (HAG) in the Department of Computer Science and Engineering at Indian Institute of Technology (ISM) Dhanbad, India. He has contributed 212 research publications in his credit and co-authored seven books and three book chapters. He has also produced 21 PhDs. He is a Senior Member of IEEE. As a recognition of his outstanding research contributions, he was awarded the Canara Bank Research Publication Award in the years 2015 and 2017. He is also among the World's top 2% (all fields) of Indian scientists in Artificial Intelligence surveyed by Stanford University, USA, during 2020–2024. His current research interests include cloud computing, edge/fog computing, the Internet of Things, and machine learning. He has served as a referee in many reputed International Journals. He has also served as a member of the Advisory Committee and Programme Committee members of several International Conferences.